The cluster module

11 min read Updated 24 Jul 2026 Topic 61 of 70

Bookmark Mark done

Why it matters

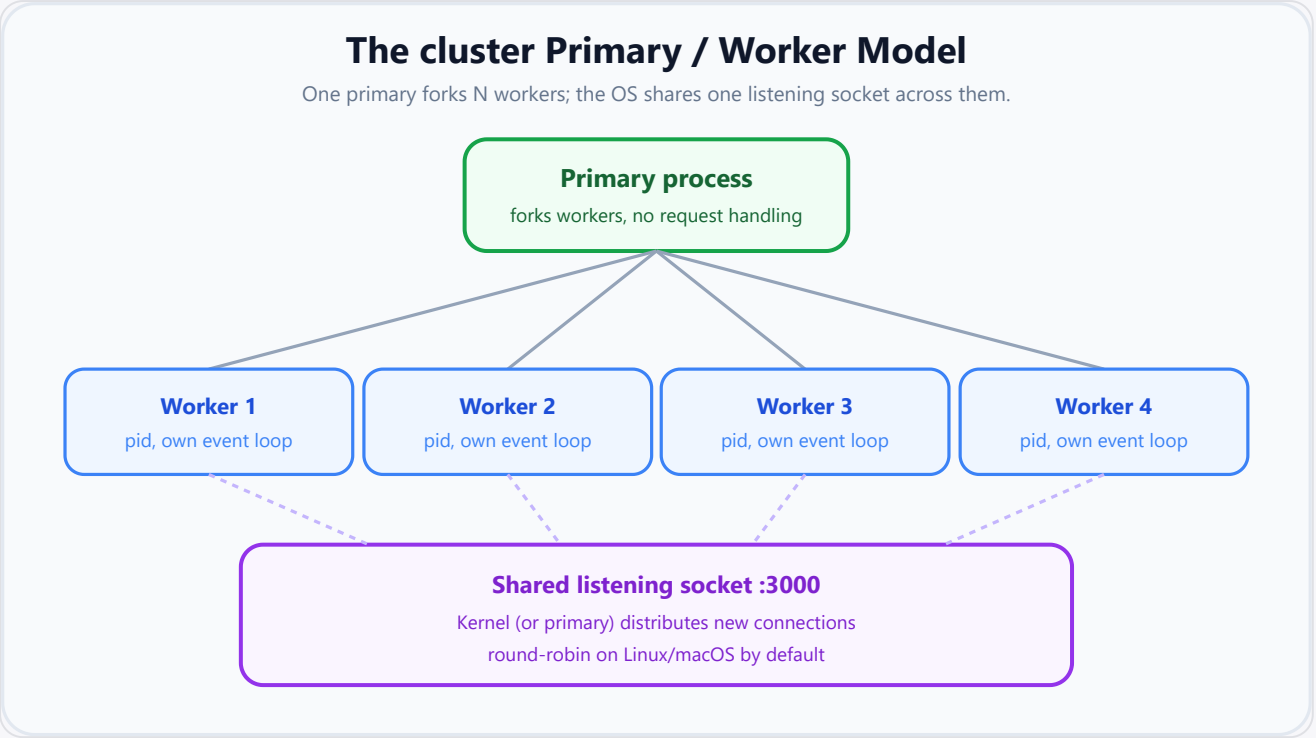
Node.js runs your JavaScript on a **single thread**. That is the defining fact of the runtime, and it has a blunt consequence: a plain Node process uses exactly *one* CPU core, no matter how many the machine has. Rent an 8-core server, run `node server.js`, and seven cores sit idle while one does all the work. Under real traffic that single core saturates, the event loop backs up, and latency climbs — while `htop` shows the box is 88% idle. This is one of the most common "why is production slow?" surprises for engineers new to Node.

The **cluster** module is Node's built-in answer. It lets you fork the process into one *worker* per core, all sharing the same server port, so incoming connections are spread across every core. For a typical I/O-bound web service, moving from one process to N workers is close to an **N× throughput** increase for zero application-code changes. In interviews, "how do you make Node use all the cores?" is a near-guaranteed question, and cluster (or a process manager that wraps it, like PM2) is the canonical answer. Knowing how it works — and its limits — separates people who have run Node in production from people who have only read about it.

The concept, simply

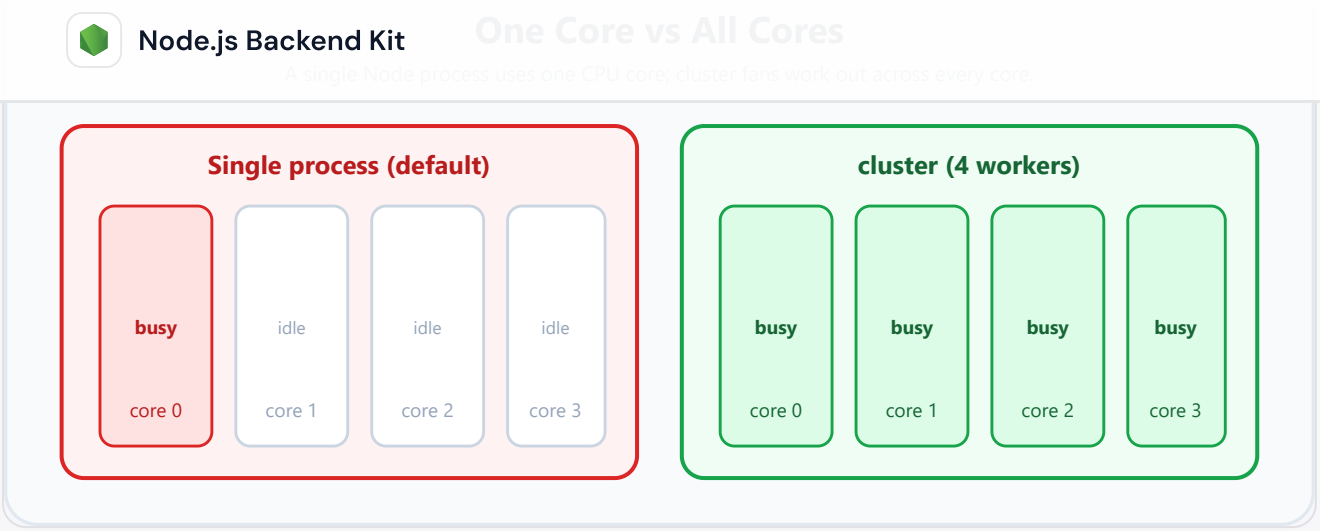
Cluster follows a **primary / worker** model (older docs and APIs say "master/worker"; the modern term is primary). One *primary* process starts first. Its only job is to **fork worker processes** and supervise them — it does not handle requests itself. Each worker is a full, independent Node process with its own memory, its own V8 heap, and its own event loop. The workers are where your actual app runs.

own socket. The primary opens the listening socket once and hands it down, so all workers accept connections from the *same* socket. The operating system (or the primary, depending on platform) then distributes new connections among them.



The primary forks one worker per core; all workers accept from a single shared listening socket, and connections are distributed among them.

The mental model to lock in: **cluster is horizontal scaling *inside a single machine***. It is the same idea as running many replicas behind a load balancer, except the "load balancer" is the kernel and the "replicas" are sibling processes sharing a port. Because each worker is a separate process, they share *nothing* in memory — no shared variables, no shared in-process cache, no shared counters. That isolation is both the safety (one worker crashing doesn't take the others down) and the main constraint (any shared state must live outside the process, e.g. in Redis).



A lone process pins one core; cluster spreads workers so every core does useful work.

How it works

When you run a cluster program, the same file executes in both the primary and every worker — the code uses `cluster.isPrimary` (aliased `cluster.isMaster` in older code) to branch. In the primary branch you call `cluster.fork()` once per core; in the worker branch you start your normal HTTP server. Under the hood `fork()` uses `child_process.fork()`, so each worker is a genuine OS process with an IPC channel back to the primary.

Connection distribution has two strategies. On Linux and macOS the default is `SCHED_RR` (round-robin): the primary accepts connections and hands each one to the next worker in turn, which spreads load evenly. On Windows the default is `SCHED_NONE`: the primary passes the listening socket to all workers and lets the OS decide, which can be uneven because the kernel tends to favour whichever worker is ready first. You can force round-robin everywhere by setting `cluster.schedulingPolicy = cluster.SCHED_RR` before forking. Round-robin distributes *connections*, not requests — with HTTP keep-alive, one connection can carry many requests, so a long-lived connection stays pinned to one worker.

Workers communicate with the primary only through **message passing** over IPC (`worker.send()` / `process.on('message')`). Messages are serialized (structured-clone by default), so you cannot pass a live object or a shared buffer through this channel and expect shared memory — each side gets a copy. This is exactly why cluster is *not* a tool for sharing CPU-heavy data structures; it is a tool for accepting more concurrent connections.

exception, an OOM kill) degrades throughput for milliseconds instead of taking the service down. This also enables **zero-downtime reloads**: on a deploy signal, fork new workers, wait for them to come up, then gracefully retire the old ones one at a time — the port never stops listening.

One subtlety: the right number of workers is usually the number of *logical* cores, which you read from `os.availableParallelism()` (added in Node 19.4; prefer it over the older `os.cpus().length` because it respects cgroup/container CPU limits). In a container capped at 2 CPUs, spawning 16 workers just creates scheduler thrash and memory bloat — each worker carries its own V8 heap.

Code examples

The canonical cluster setup: primary forks one worker per available core and respawns on death; workers run an ordinary HTTP server.

JAVASCRIPT

Copy

```
// server.js
import cluster from 'node:cluster';
import http from 'node:http';
import { availableParallelism } from 'node:os';
import process from 'node:process';

if (cluster.isPrimary) {
  const cpus = availableParallelism();          // respects container limi
  console.log(`Primary ${process.pid} forking ${cpus} workers`);

  for (let i = 0; i < cpus; i++) cluster.fork();

  cluster.on('exit', (worker, code, signal) => {
    console.warn(`Worker ${worker.process.pid} died (${signal || code}); r
    cluster.fork();                             // keep the pool full
  });
} else {
  // Each worker is a full Node process with its own event loop.
  http.createServer((req, res) => {
    res.end(`Handled by worker ${process.pid}
```

```
  console.log(`Worker ${process.pid} listening on :3000`);
}
```

Primary and workers can talk over IPC. Here the primary keeps a global request tally by collecting counts the workers report — the kind of aggregation you cannot do with a shared variable.

JAVASCRIPT Copy

```
// Aggregating per-worker metrics in the primary
if (cluster.isPrimary) {
  let total = 0;
  for (let i = 0; i < availableParallelism(); i++) {
    const worker = cluster.fork();
    worker.on('message', (msg) => {
      if (msg.type === 'hit') {
        total++;
        if (total % 1000 === 0) console.log('total requests:', total);
      }
    });
  }
} else {
  http.createServer((req, res) => {
    process.send({ type: 'hit' });              // report up to the primar
    res.end('ok');
  }).listen(3000);
}
```

Graceful, zero-downtime worker reloading — retire workers one at a time so the port always has capacity behind it.

JAVASCRIPT Copy

```
// Rolling restart of workers without dropping the port
async function reload() {
  const workers = Object.values(cluster.workers);
```

```
      replacement.once('listening', () => {
        worker.disconnect();                    // stop new conns to old w
        worker.once('exit', resolve);           // wait until it drains +
      });
    });
  }
}
process.on('SIGUSR2', reload);                  // trigger on a deploy sig
```

Common mistakes & gotchas

- **Assuming shared memory.** Workers are separate processes. An in-memory cache, rate-limit counter, or session map in one worker is invisible to the others. Round-robin means consecutive requests from the same user can land on different workers — so a login stored in worker A's RAM looks logged-out on worker B. Put shared state in Redis/DB or a sticky session layer.
- **Thinking cluster fixes CPU-bound code.** Cluster multiplies your capacity for *concurrent I/O*. If a *single* request does heavy synchronous computation, it still blocks its worker's one thread; cluster just means fewer requests are affected at once. For per-request CPU work, reach for worker_threads instead.
- **Forking too many workers.** More workers than cores causes context-switch thrash and multiplies memory (each worker = a full V8 heap). In containers, `os.cpus().length` reports host cores, not your CPU limit — use `availableParallelism()`.
- **WebSockets / sticky sessions.** Round-robin distributes the initial connection, but stateful protocols (Socket.IO with in-memory adapters, long polling) assume a client stays on one worker. Without sticky routing and a shared adapter (e.g. the Redis adapter), messages get lost across workers.
- **No respawn logic.** Forgetting the `'exit'` handler means every crashed worker permanently reduces capacity until the whole service is restarted.
- **Sharing a DB connection object across the fork.** Connections are per-process; each worker must open its own pool. You cannot inherit a live socket handle as a usable

Best practices

- **Prefer a process manager in production.** PM2 (`pm2 start app.js -i max`) or the container orchestrator's replicas wrap cluster's fork/respawn/reload logic so you don't hand-roll it. Many teams run one process per container and let Kubernetes do the horizontal scaling instead of clustering inside the container.
- **Keep workers stateless.** Externalize sessions, caches, counters, and locks to Redis or the database. Stateless workers can be killed and respawned freely.
- **Size workers to available parallelism** via `availableParallelism()`, and leave headroom for the primary and OS.
- **Implement graceful shutdown per worker** — on `disconnect` / `SIGTERM`, stop accepting, drain in-flight requests, then exit (see the graceful-shutdown topic).
- **Log the worker pid** in every log line so you can trace which worker served a request.
- **Don't cluster CPU-bound single requests** — profile first; if one request pins a core, offload to worker_threads or a queue.

⭐ Interview lens

Q: Why does a Node process only use one CPU core, and how does cluster change that?

A: Your JavaScript runs on a single main thread (libuv has a thread pool for some I/O, but *your* code is single-threaded). One process therefore pins one core. Cluster forks one worker process per core, each with its own event loop, all sharing one listening socket, so connections spread across every core — roughly N× throughput for I/O-bound work.

Q: Do cluster workers share memory?

A: No. They are independent OS processes with separate heaps. They communicate only via IPC message passing, and messages are serialized copies. Any shared state must live outside the process, e.g. Redis. (Trap: interviewers wait

Q: How are incoming connections distributed to workers?
A: Default is round-robin (`SCHED_RR`) on Linux/macOS — the primary accepts and hands each connection to the next worker. Windows defaults to letting the OS decide (`SCHED_NONE`), which can be uneven. Note it distributes *connections*, not requests, so keep-alive pins many requests to one worker.

Q: Does cluster help with a CPU-bound endpoint?
A: Only indirectly. Each worker is still single-threaded, so a heavy synchronous request blocks its worker. Cluster reduces how many requests are affected at once but doesn't speed up the computation. For genuinely CPU-heavy work per request, use worker_threads or a job queue. (Common trap.)

Q: cluster vs running multiple containers behind a load balancer — when each?
A: Same idea at different layers. Cluster maximizes a single machine's cores cheaply, in-process. Multiple containers/replicas add cross-machine scaling, isolation, and independent rolling deploys. Many modern setups run one Node process per small container and let the orchestrator scale horizontally — simpler ops, no in-process supervisor to maintain.

Q: How do you achieve zero-downtime reloads with cluster?
A: On a reload signal, fork new workers, wait for each to emit `listening`, then `disconnect()` old workers one at a time so they finish in-flight requests before exiting. The shared listening socket stays open the whole time, so the port never drops.

Cheat-card

- **Primary** forks + supervises (no request handling); **workers** run your server, share one listening socket.
- Branch on `cluster.isPrimary`; scale to `os.availableParallelism()` (container-aware).
- Distribution: **round-robin** (Linux/macOS default), by connection not request.
- **No shared memory** — IPC copies only; externalize state to Redis/DB.
- Respawn on `'exit'`; graceful reload = fork new, `disconnect()` old one-by-one.
- Great for I/O concurrency; **not** a fix for CPU-bound single requests (→ worker_threads).
- In production, usually let PM2 (`-i max`) or the orchestrator manage it.

QUICK CHECK — no marks, just for you.

1. Why does a plain Node process only use one CPU core, and how does the cluster module change that?

- ○ Your JavaScript runs on a single thread, so one process pins one core; cluster forks one worker process per core, all sharing one listening socket
- ○ Node deliberately throttles to one core; cluster removes the throttle so a single process uses all cores
- ○ The event loop uses every core already; cluster only adds logging
- ○ One process uses one core because of a memory limit; cluster raises that limit

Check answer

2. Do cluster workers share memory?

- ○ Yes, they all share one V8 heap and can use a common in-memory cache

Yes, but only for variables declared before cluster.fork()

- No — they are independent OS processes with separate heaps and communicate only via IPC message copies; shared state must live in Redis/DB
- No, but they automatically synchronise module-level variables

Check answer

3. Does adding cluster workers speed up a single CPU-bound request?

- Yes, the heavy computation is automatically split across all workers
- Yes, because each request is divided among the available cores
- No, you must use async/await instead to unblock the computation
- No — each worker is still single-threaded, so a heavy synchronous request blocks its worker; cluster only limits how many requests are affected at once

Check answer

‹ Previous
Mocking & Coverage

Next ›
Worker threads

LEARN MORE

ARTICLES & DOCS

- Node.js Cluster module
  nodejs.org
- Node.js Worker Threads (cluster vs threads)

nodejs.org

Node.js Backend Kit

VIDEOS



Scaling your Node.js app using the "cluster" module

Software Developer Diaries

Hand-picked references for this lesson.